

The current State of the Art in Polygon Clipping

Technical University of Darmstadt
MMC Seminar, Summer Term 2023

Jan Schulz-Nigmann
jan.schulz-nigmann@stud.tu-darmstadt.de

Abstract—The paper evaluates different polygon clipping algorithms and contrasts how well they perform in terms of complexity and runtime. We look into algorithms such the Cohen-Sutherland, Cyrus-Beck, Dimri, Liang-Barsky, Vatti, Martinez-Rueda, Sutherland-Hodgman, Weiler-Atherton, Slater-Barsky, and Greiner-Hormann. We evaluate each algorithm's advantages and disadvantages and offer empirical support based on benchmarking experiments. Degenerate intersections and self-intersecting polygons are both effectively handled by the Greiner-Hormann algorithm. Although significantly slower, the Foster-Hormann algorithm performs best with polygons of a median size. The Maillot algorithm outperforms Sutherland-Hodgman. Efficiency-wise, Slater-Barsky surpasses Sutherland-Hodgman. Line clipping-based algorithms and novel methods like the point-based Dimri algorithm are compared. The paper also analyses various libraries for polygon clipping, including General Polygon Clipper (GPC), CGAL, Shapely, Boost.Geometry, and Clipper. Each library makes use of many algorithms, including modified versions of Vatti and Weiler-Atherton. We give benchmarking results to assess these libraries' performance while taking runtime, scalability, and handling of complicated polygons into account. For big clips, the Boost.Polygon library performs better than GPC and PolyBoolean.

Keywords—Polygon Clipping, Clipping Algorithms, Clipping, Polygon, Computational Geometry, Clipping Libraries

I. INTRODUCTION

Cutting an object at the border of a window or other designated area of the screen is the process of clipping. The window may represent a specific form of polygon. Clipping is classified into two types: line clipping and polygon clipping.

The Cohen-Sutherland [1], Cyrus-Beck [2], and Liang-Barsky [3] Algorithms are three examples of line clipping algorithms that can be used to tackle problems when the only element that must be cut is a set of lines, and the clipping window is a rectangle. The scanline algorithm [4] [5] [6] is the most common when the window is an arbitrary polygon.

Polygon clipping essentially is when the object is a polygon (also called the subject polygon). In this study, these polygon-clipping algorithms will be reviewed and compared. The term "window polygon" is used to describe the clip polygon. See Figure 1

These polygons can be convex (all inner angles are less than 180 degrees) or concave (at least one inner angle is greater than 180 degrees), simple (the polygon's edges don't intersect) or complex (at least two of the polygon's edges intersect).

We aim to provide an overview of the state-of-the-art in polygon clipping in this study.

The methodology will be introduced in Section II. A summary of the many algorithms throughout years of development

is given in Section III. In Section IV, various algorithms are contrasted with one another. In Section V, an overview of the most widely used clipping tools is provided.

The use of polygon clipping is widespread across several academic fields. It forms the basis for rendering complicated scenes in computer graphics, enabling hidden surface removal, anti-aliasing, and real-time visualisations. Geographic information systems heavily rely on polygon clipping for spatial analysis, such as overlay operations, terrain analysis, and map generalization. Additionally, polygon clipping techniques are used in robotics and computer vision to address issues with motion planning, object recognition, and sensor data fusion.

Polygon clipping algorithms can be utilised in 2D CAD systems such as KiCad (Electronic Design Automation) to optimise Printed Circuit Board (PCB) ground plane design. Polygon clipping ensures adequate connectivity and optimal grounding in PCB ground plane design. Polygon clipping methods provide rapid geometric computations, precise shape manipulation, and sophisticated polygon interoperation. These applications demonstrate the importance of polygon clipping in driving innovation, boosting design efficiency, and ensuring the effective completion of geometrically complex tasks across multiple domains.

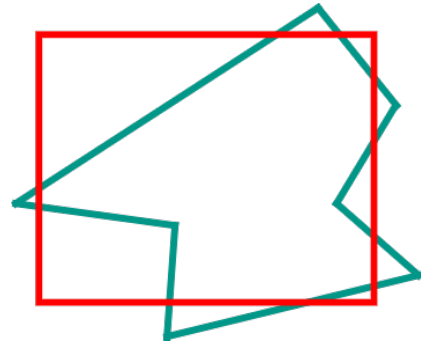


Fig. 1: The red polygon is called the clip window or clip polygon (does not need to be a rectangle, can be an arbitrary polygon) and the blue polygon is called the object or subject polygon. A more complicated case can be found in Figure 2.

II. METHODOLOGY

The research and analysis of polygon clipping algorithms, performance comparisons, and the assessment of various polygon clipping libraries and tools were all part of the

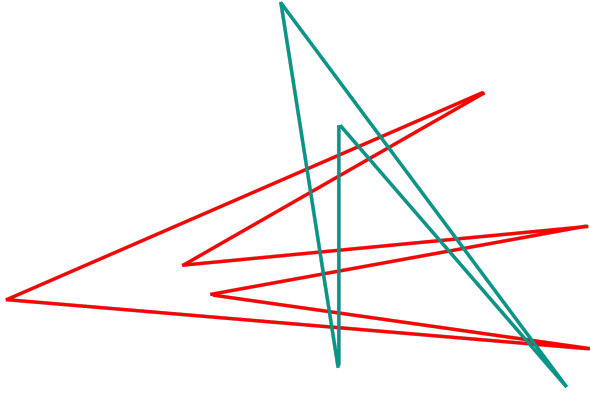


Fig. 2: A typically complicated polygon clipping case. Worst case for the intersections of two polygons.

methodology used in this paper. The strategy is described in the following way:

- 1) Online Research: To fully understand polygon clipping and identify the available algorithms for this issue, a significant amount of online research was conducted in the first phase. This step was designed to acquaint us with the subject and lay the groundwork for further investigation.
- 2) Algorithm Selection: After gaining an understanding of the available algorithms, we performed a careful selection process. This required assessing the applicability, importance, and potential of each algorithm. We investigated the papers written by those who proposed these algorithms in order to make educated deductions. We gained an understanding of the fundamental ideas, methods, and benefits of each algorithm by examining these articles.
- 3) Summary of Findings: After thoroughly studying the chosen algorithms, we went on to highlight the key findings from our study. This required distilling the data from the algorithm papers, emphasising their key characteristics, and contrasting their advantages and disadvantages.
- 4) Performance Comparison: To evaluate the performance of the selected algorithms, we researched existing literature that offered comparisons between different algorithms. We began our search with the articles from Step 3 and then broadened it to include other sources that provide performance evaluations and benchmarks. This step is intended to pinpoint the algorithm(s) that performed best in terms of effectiveness, precision, or other important factors.
- 5) Analysing Different Polygon Clipping Libraries and Tools: In addition to comparing different algorithms, we also looked into various polygon clipping libraries and tools. As a starting point for this investigation, we used a work by Simonson. [7] We undertook further research

to examine the capabilities, features, and performance of several polygon clipping libraries and tools in light of Simonson's work. In this step, documentation was reviewed, case studies were examined, and practical applications were taken into account.

- 6) Performance Evaluation of Libraries and Tools: Using the data we had acquired, we went on to assess how well the libraries and tools we had looked at for polygon clipping performed. This evaluation involves contrasting their usability, speed, and any other pertinent elements. We may perhaps find the best libraries or tools for specific use cases by using the data we acquired from multiple sources to make informed judgements.

By employing this process, we were able to develop a solid understanding of polygon clipping algorithms, compile a list of their key features, assess how well they performed, and survey the ecosystem of accessible libraries and tools. These procedures gave us a thorough comprehension of the subject, which allowed us to give an informed analysis.

III. ALGORITHMS THROUGHOUT YEARS OF DEVELOPMENT

A. Sutherland-Hodgman (1974)

One of the earliest polygon clipping methods is the Sutherland-Hodgman [8] algorithm, which was first proposed in 1974 and is based on the Cohen-Sutherland line clipping algorithm. The method is still widely used today despite being close to 50 years old. Later, several alternative algorithms will take advantage of the concept of extending a line clipping technique to furthermore clip polygons. In addition, the Sutherland-Hodgman algorithm is frequently used as a base to compare or extend. Each polygon edge or vertex is successively clipped by each clipping zone edge in the Sutherland-Hodgman algorithm. At each edge, the algorithm recursively calls itself.

The algorithm contains a few flaws as well. The resulting, clipped polygon may have coincident (i.e., overlapping) edges if the subject polygon was concave at vertices outside of the clip window. The clip window can only be convex (but this only with an extension, normally the algorithm only supports rectangular clip windows). For rendering, this is acceptable, but not for other uses like computing shadows. Additionally, the approach does not handle degeneracies and holes; nonetheless, solutions exist.

B. Weiler-Atherton (1977)

A more all-encompassing method that can be used to compute intersections between two polygons is the Weiler-Atherton [9] algorithm. Despite the algorithm's considerable power, it is not well suited for use with a 2D rectangular clipping window. Kilgour [10] has published an improved version of this method for rectangular clipping windows that also makes use of the vector clipping's trivial reject information.

The algorithm operates by repeatedly iterating through a polygon's vertices to determine whether or not they are within or outside of another polygon. A vertex is included in the

output list if it is within. If the vertex is outside, intersection points are found between the polygon edges. When the polygons' roles are reversed, this process is repeated. The clipped polygons that represent the intersection of the original polygons are included in the output list that is produced. Using visible surface determination and Z-ordering, this approach can also be applied in three dimensions.

C. Liang-Barsky (1983)

The polygon clipping algorithm proposed by Liang and Barsky [11] is based on the parametric equation of the polygon edges. Four scalar parameters are possible from one edge. This algorithm generates the required turning points¹ by creating a classification based on the contribution of each parameter. The authors claim that this approach is nearly twice as fast as the Sutherland-Hodgman algorithm, although it has the disadvantage that it necessitates using floating point calculations.

D. Maillot (1992)

The fundamental intersection calculations in Maillot's [12] proposed algorithm are performed using the Sutherland-Cohen Line clipping algorithm. The advantage of this strategy is that it is the fastest for most applications. The Sutherland-Cohen line clipping algorithm was chosen since it is most likely the most effective technique for trivial acceptance and rejection cases, both of which are the most typical scenarios in visualisation. This approach can be applied to a larger range of applications because it can be implemented using either integer or floating-point arithmetic. It is simpler to optimize the algorithm in the case of polylines, which is a case that closely resembles that of a polygon border, compared to optimising the algorithm for one vector at a time.

E. Vatti (1992)

The sorts of polygons that can be used as subjects or clips are not constrained by the Vatti [13] algorithm. It is possible to process even complicated (self-intersecting) polygons and polygons with holes. The algorithm typically only works in 2D space.

According to the Vatti algorithm, subject and clipping polygon edges are processed sequentially, starting at the bottom and moving up. By using scanlines, which are fictitious horizontal lines that travel through each vertex of the involved polygons, this sweep line strategy splits the issue space. The spaces between neighbouring scanlines are outlined by the so-called scanbeams. The algorithm adds intersection points within these scanbeams to the solution polygons as it processes each scanbeam in turn, beginning with the lowest scanbeam.

F. Slater-Barsky (1994)

The Slater-Barsky [14] algorithm can clip planar polygons to a rectangular clipping window and is based on the space

subdivision paradigm.² When a polygon is clipped, additional vertices are created where its edges cross the clipping region's bounds. The procedure requires traversing the polygon's edges in an ordered manner in search of these intersections. Each edge is evaluated by the algorithm against the entire clipping region. The output polygon is created by recording the intersection points and corner cells after the algorithm generates a sequence of cells for each edge. This algorithm's nomenclature depicts the movement of edges between cells as well as their orientation. The Slater-Barsky algorithm is an extension of the Liang-Barsky line clipping algorithm.

G. Greiner-Hormann (1998)

Both self-intersecting and non-convex polygons can be handled by the Greiner-Hormann [15] algorithm. The algorithm's cornerstone is the definition of a polygon's "inside" based on its winding number. The even-odd rule states that areas with an odd number of windings are considered to be inside the polygon. As a consequence, the algorithm receives two lists of polygons as inputs. There are three phases to the algorithm. Pairwise crossings between edges are computed in the initial stage. At the sites where the two polygons intersect, new vertices are added, and the intersection polygon points to the counterpart in the other polygon. These crossings are designated as entry or exit intersections in the second phase, resulting in a classification of inside or outside a polygon. The algorithm propagates over these vertices in the third phase, computing the final result.

As long as there is a sufficient pairwise intersection mechanism, the algorithm can handle arbitrary parametric curves as segments and is not limited to polygons. The Greiner-Hormann algorithm's inability to handle degeneracies such as similar edges or intersections precisely at a vertex is one of its main drawbacks. Greiner and Hormann recommend removing the vertices in their original study by perturbing them.

H. Zhang-Sabharwal (2002)

Zhang and Sabharwal [16] present a new polygon clipping algorithm that is simple, compact, efficient, and easy to implement. Because it does not rely on classifying the edges as entry or exit points, it does not necessitate sorting edges before clipping. The polygon clipping algorithm extends their proposed line clipping algorithm. The orientation of the clip and subject polygons has no bearing on how the algorithm handles any special cases. The subject polygon's clipped edges are shown, and the clip window's edges are then scrutinised to create the clipped polygon(s). It is required to decide whether a vertex of the clip polygon should be considered inside or outside the subject polygon in the critical case where it lies on an edge of the subject polygon. The important case must be handled specifically in this situation, but once it is, the rest is straightforward. The regions enclosed by the subject

¹A turning point represents the location where a line segment intersects the boundaries of the clipping window, indicating a transition between the inside and outside of the clipping region.

²The space subdivision paradigm is a concept used in computer graphics and computational geometry to divide a space into smaller regions for efficient processing.

polygon and clip polygon are closed and the boundaries of clipped regions are polygons.

I. Simonson (2010)

Simonson [7] describes a new polygon clipping algorithm that computes the topology of several different polygon sets in a single sweepline pass. The approach computes the sum of overlapping polygon counts as a Riemann integral using a sweepline and is based on a vertex representation of polygon data. The algorithm computes a Riemann-Stieltjes integral by snapping intersection points to the integer grid to approximate the Riemann integral. The algorithm scales nearly linearly. The approach is resistant to all types of degeneracy in the input polygons' geometry. The approach requires integer input and output polygon coordinates, but it is numerically resilient without compromising efficiency. According to the author, a polygon is a collection of vertices and edges, where each edge denotes a vector and a direction of winding that can be either clockwise or counterclockwise. They categorise the inside and outside of a polygon using a variation of the non-zero winding rule (Similar to the Greiner-Hormann algorithm).

J. Martinez-Rueda (2013)

An algorithm for polygon clipping was developed by Martinez and Rueda [17] that makes use of the plane sweep technique. Events are produced at the ends of polygon edges and at intersection locations as a vertical line sweeps over the plane. The algorithm separates the edges at the points of intersection and chooses the edges that belong to the resulting polygon based on whether they are inside or outside the other polygon. The algorithm operates in two stages: the first subdivides the edges and chooses the edges that belong to the resulting polygon, and the second builds the resulting polygon out of the selected edges. Additionally, Martinez and Rueda go over some of the degenerate cases that can occur, such as overlapping and vertical edges, and how to address them.

K. Foster-Hormann (2019)

The Greiner-Hormann algorithm for clipping two polygons is extended by the Foster-Hormann [18] algorithm. A re-designed labelling technique is used in the extension to identify and mark a subset of all intersection vertices as crossing intersections using local orientation checks. The tracing phase of the algorithm stays essentially the same, and these are then tagged alternatively as entry and exit points, just like in the original Greiner-Hormann algorithm. The updated algorithm includes proper handling of degenerate intersections.

L. Dimri (2022)

A thorough approach for 2D polygon clipping is the Dimri [19] algorithm. The proposed polygon clipping technique calculates the point of intersection of the edges' coordinates. In order to produce the clipped polygon, this technique also determines the genuine intersection points and computes the limits of the clip window. The proposed approach makes use of the idea of point clipping and the parametric equation

of polygon edges. The suggested approach is more practical mathematically and has applications in graphics and other fields. The algorithm's restriction is that it can only be used for 2D rectangular clip windows.

IV. COMPARISON

There are countless unique attributes that polygon clipping methods can be evaluated on. We chose the following qualities for this study because they are crucial for the creation of toolpaths for computer numerical control (CNC) systems. The clip window, the subject polygon, handling of holes, loose polygons, and degeneracies are all common traits. Efficiency and scalability (big polygons and plenty of little polygons) are particularly intriguing to us. Convex, concave, simple, and complex subject polygons and clip window types were already described in Section I. A group of arbitrary, non-overlapping polygons is referred to as a loose polygon. They can be of any size, have any number of sides, and may or may not cross over one another. Polygons with holes are polygons whose bounds include one or more cut-out sections. These cut-out regions can have any shape. Degeneracies are unique situations or anomalies in polygons that call for a particular treatment. Instances like zero-area polygons (points or collinear vertices), coincident vertices (overlapping), or edges with zero length can all be examples of this. These scenarios need to be handled correctly to guarantee accurate polygon clipping. Since they define the characteristics and constraints of the processed polygons, understanding these ideas is crucial to formulating a qualitative evaluation of polygon clipping algorithms. This knowledge makes it possible to perform trimming techniques correctly and ensures consistent results. The challenge with this is that the various algorithms can exploit various characteristics of the different programming languages, as well as be further optimised and hardware accelerated. Therefore, we rely on the references and the claimed efficiency and scalability of their authors for our evaluation. Foster-Hormann [18] and Martinez-Rueda [17] provide a lot of information about their proposed algorithms compared to the Vatti and the Greiner-Hormann algorithm. To further analyse this, we shall refer to Martinez-Rueda's timings in Table I. The precise timings of

Number of vertices	Greiner	Vatti	Martinez	Number of Intersections
$25,786 \times 18,065$	0.02	0.11	0.04	10,405
$124,636 \times 55,648$	6.1	4.2	0.66	33,786
$223,834 \times 175,629$	120.4	35.3	1.3	91,034

TABLE I: Execution times of intersection operations (in seconds) from [17]. Martinez and Rueda clipped polygons representing continents, the first number of vertices is the subject polygon and the second is the clip window. The last column is the number of intersections between the edges of the polygons.

Martinez and Rueda were verified by Foster and Hormann. They also claim that their extension of the Greiner-Hormann algorithm is marginally slower because the intersection phase typically consumes more than 80% of the running time,

making the minimal overhead caused by the new labelling step irrelevant. If $k \in O(nm)$, where k is the number of intersections of polygon P and polygon Q , n is the number of edges of P , and m is the number of edges of Q , then the Greiner-Hormann algorithm requires $O(nm)$ operations. Vatti's approach needs $O((n + m + k)\log(n + m))$. Martinez-Rueda's approach is quicker than Greiner-Hormann's approach if $k \in O(n+m)$ and it can handle degenerate intersections. For totally random polygons with many self-intersections, the Greiner-Hormann and Foster-Hormann algorithms perform better than the plane-sweep-based methods (Vatti, Martinez-Rueda), simply because the latter does not need to be computed. In addition, the Foster-Hormann algorithm is the fastest when applied to moderately sized polygons ($mn \leq 1\,000\,000$, $k \leq 10\,000$), outperforming Greiner-Hormann, Vatti, and Martinez-Rueda. This is most likely due to the exceptionally effective labelling and tracing stages, both of which take less than 1 ms.

A C implementation of the Sutherland and Hodgman method and a C implementation of the Liang and Barsky algorithm were used by Maillot as comparison examples for his algorithm. The runtime of each implementation on 100000 consecutive calls of the specified figure is displayed by Table II

Input data	Suth. & Hodg.	Liang & Barsky	Maillot	
	Float	Float	Float	Int
Figure 3-a	48.0	23.0	22.6	21.3
Figure 3-b	24.0	8.7	4.6	3.6
Figure 3-c	69.2	40.2	33.1	23.4
Figure 3-d	22.7	7.6	4.2	3.1
Figure 3-e	24.6	7.6	7.5	3.9

TABLE II: Execution times of 100000 consecutive calls (in seconds) from [12]

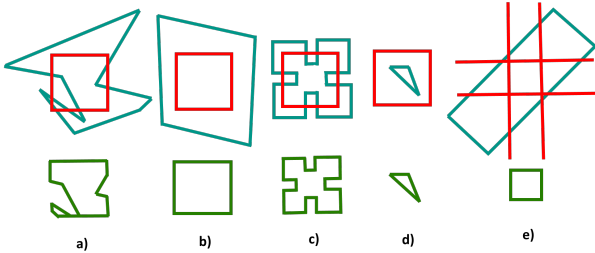


Fig. 3: These are the Polygons on which Maillot performed their tests.

Dynamic memory allocation was utilised by the Sutherland-Hodgman algorithm to deal with polygons of various complexities. Dynamic memory allocation could be avoided by restricting the number of vertices, cutting down on the time required for memory operations. The graphical output from all three algorithms was the same because they were all built with the same degree of optimisation. Maillot demonstrated that the Sutherland-Hodgman algorithm spent a lot of time managing memory and performing memory copy operations. The algorithm's performance was therefore 1.5 times better

after dynamic memory allocation was removed, but it was still inferior to the Maillot algorithm. Maillot assessed the algorithms' performance with a focus on maximising effectiveness for trivial acceptance or rejection cases. The Sutherland-Cohen approach for line clipping made sure that polygons that were trivially rejected produced better results than those that were trivially accepted.

Simonson compares his suggested approach to a few popular clipping libraries, therefore in Section V, we will assess the effectiveness and complexity.

The Weiler-Atherton has been reported to have an $O(n^2)$ [20] runtime complexity. Weiler and Atherton simply emphasise that their approach is better equipped to use the provided hardware to accelerate the calculation than prior algorithms. They do not specify a projected complexity or runtime.

The weighted operation score is used by Slater and Barsky to compare their approach with the Sutherland-Hodgman algorithm. The term "weighted score" refers to a score that accounts for the computing expense of each algorithmic action. This means that a weight or value that reflects the relative computational complexity of each operation (such as addition, subtraction, division, multiplication, etc.) is allocated to each operation. The weighted score is then determined by adding the weighted values of all the algorithm's operations. Simply explained, an unweighted score is a score that does not take into account the computing expense of particular operations. Without giving each action a distinct weight or value, it simply counts the total number of operations the algorithm executed. In both the weighted and unweighted cases, this results in scores of four and eight, respectively. According to Slater and Barsky, their algorithm has a weighted score of 1.25 and an unweighted score of 2.5. In the worst situation, the Slater-Barsky algorithm outperforms the Sutherland-Hodgman algorithm, which has a weighted score of 64 and an unweighted score of 64, compared to a weighted score of 30 and an unweighted score of 40.

Only data comparing Zhang and Sarbharwal's line clipping method to the Cohen-Sutherland, Cyrus-Beck, and Liang-Barsky line clipping algorithms are presented. Since their polygon clipping extension is based on line clipping methods, we assume that it scales similarly to the Liang-Barsky polygon extension or other polygon algorithms. We will not go into more depth with the numbers because they are not particularly representative and the algorithm is a niche algorithm.

A relatively new approach, the Dimri algorithm is based on the idea of point clipping and the parametric equation of polygonal edges. Dimri asserts that his approach is technically less expensive than polygon clipping algorithms based on line clipping, however, he provides no evidence to support this assertion. This claim needs to be further examined in future studies.

Table III gives an overview of the algorithms.

A. Limitations

When evaluating polygon clipping algorithms, it is important to keep in mind that many of them have restrictions and particular requirements. We'll provide a brief description of one of these requirements. In some situations, the polygon's edges need to be arranged clockwise or anticlockwise [9]. Additional processing, which frequently does not contribute to the runtime or complexity of the algorithm itself, is necessary to verify that the edges are accurately represented in the input list of edges. Therefore, it is difficult to compare how well various algorithms perform in the real world.

V. LIBRARIES

The JTS Topology Suite (JTS) is an open-source Java library for working with spatial data and performing geometric operations. It offers a collection of reliable and effective spatial operations handling methods, including point-in-polygon checks, polygon intersection, buffer generation, and more. The JTS, which was introduced in 2001, offers a full selection of polygon clipping techniques through its Geometry class and associated utility classes. Self-intersecting polygons, complicated polygons with holes, and polygons with degenerate situations can all be handled by these techniques. The library offers solid algorithms that guarantee reliable and accurate results and support both 2D and 3D geometries. The JTS library uses a modified version of the Vatti algorithm [21].

A strong and well-known open-source library for geometric operations is GEOS (Geometry Engine - Open Source) [22]. It was first made available in 2002 and is a port of the JTS library. For manipulating and analysing geometric objects, including points, lines, polygons, and more, it offers a powerful collection of operations. A vital part of many well-known geographic information systems (GIS) and geographical databases, GEOS is developed in the C++ programming language. The extensive collection of polygon clipping functions offered by GEOS enables the user to carry out several operations on polygons, including intersection, union, difference, and symmetric difference. Both simple and complex polygons, including those with holes or self-intersections, can be subjected to these processes. The robustness of GEOS for polygon clipping is one of its key benefits. It deals with degenerate situations, including polygons with overlapping edges or vertices, and makes sure the final geometry is accurate and consistent. GEOS is effective for handling huge datasets since it provides a variety of spatial indices and methods to optimise the clipping procedure. Therefore, the Vatti algorithm is also used by this library.

A robust Python module for geometric operations and manipulations is the Shapely [23] library, which was published in 2008. It offers a collection of instruments for manipulating and examining planar geometric objects, including points, lines, and polygons. The widely used geometric library GEOS, which provides the computational engine for carrying out geometric operations, is the foundation upon which Shapely is based. Therefore, the Vatti algorithm is also used by this library.

A robust C++ library called Boost.Geometry offers a wide range of geometric algorithms and data structures. Numerous functions, including point and vector manipulations, geometric algorithms, spatial indexing, and polygon operations, are available with Boost.Geometry. Support for polygon clipping is one of Boost.Geometry's standout features. Concerning both basic and complex polygons, Boost.Geometry offers effective algorithms that smoothly handle degenerate instances and accommodate edge cases. Other polygon-related operations supported by the library include convex hull computation, polygon simplification, centroid calculation, and others. Due to these features, Boost.Geometry is a useful tool for managing geometric data and resolving a variety of geometric issues. It was published in 2010, and the library's creator claimed in a blog post that he had modified the Weiler-Atherton algorithm [24]. The Boost.Polygon library, which is a component of the Boost library, was developed by Simonson and uses his algorithm [7].

The General Polygon Clipper (GPC) library was published in 1997 by Alan Murta [25]. The original developer is not currently distributing it, although it is openly accessible on Git Hub under the MIT Licence. A modified version of the Vatti algorithm is utilised by the GPC [26].

A robust C++ library called CGAL (Computational Geometry Algorithms Library) [27] provides a collection of effective and dependable methods for resolving a variety of computational geometry issues. It provides a variety of tools for working with geometric data structures, geometric computation algorithms, and visualisation techniques. It was first published in 1997 and makes no mention of the computations' method of choice.

Applications requiring polygon clipping and offsetting, such as computer graphics, geographic information systems (GIS), computational geometry research, and computer-aided design (CAD) systems, have made extensive use of the initial Clipper (Clipper 1) library, as it handles polygon operations precisely, supports both integer and floating-point coordinate systems, and has a straightforward API. It can effectively and robustly handle polygons with holes and self-intersecting polygons. It was created and published in 2010 by Angus Johnson and makes use of a modified Vatti algorithm. The second version of Clipper, known as Clipper 2, was introduced in 2022 and completely replaces Clipper 1 [28]. It is more accurate and reliable because of its enhanced performance and better support for floating-point coordinate systems. Additionally, Clipper 1's feature set is substantially expanded by it.

A. Performance

Simonson performed benchmarking tests to evaluate their polygon clipping Boolean approach against the General Polygon Clipper (GPC), PolyBoolean, and Computational Geometry Algorithms Library (CGAL) libraries. All of the implementations were written in C++ using the same compiler and optimisation settings, and the benchmarks were run on a particular hardware configuration. Axis-parallel/rectilinear (Manhattan) polygon data, which is frequently used in VLSI

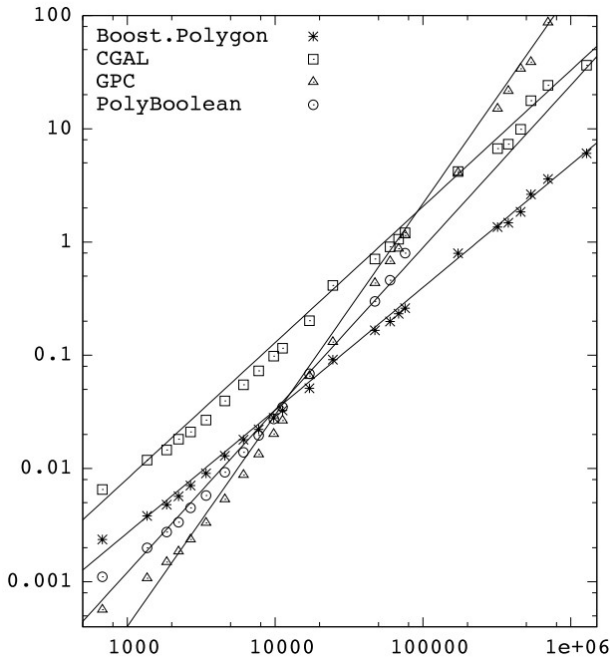


Fig. 4: General polygon clipping scalability comparison from Simonson [7]. The vertical axis represents the runtime in seconds using a logarithmic scale. The horizontal axis represents the number of input and output vertices combined using a logarithmic scale.

applications, and general polygon data were both used in the two trials. Two sets of tiny triangles or rectangles that were randomly arranged within a square region made up the input data. The authors utilised their implementation to combine the input sets into appropriate polygons for clipping by the other methods because the benchmarked algorithms needed disjoint input polygons. As a result, both the input and output polygon sets contained several holes.

According to the findings, which are represented in Figure 4, GPC was the fastest for small clips but became the slowest for clips with more than a million vertices.

For such extensive clips, the authors’ approach (labelled Boost.Polygon) was the fastest. With empirical scaling factors of $O(n^{1.87})$ and $O(n^{1.43})$, respectively, GPC and PolyBoolean scaled slower than a method with $O((n + s)\log(n))$ runtime complexity. The empirical scaling factor for CGAL was $O(n^{1.20})$. The authors pointed out that the empirical scaling of their implementation, which was $O(n^{1.08})$, was nearly as large as what might be anticipated from an algorithm with $O((n + s)\log(n))$ runtime complexity. Their algorithm became the fastest at the crossover point of 10,000 vertices, completing the intersection operation very quickly. GPC, on the other hand, required far more time; for a million vertices, it took 1300 seconds as a result of swapping to disc. The following bigger data point also resulted in a memory allocation exception from GPC. Due to numerical robustness difficulties with PolyBoolean that resulted in errors or crashes in some

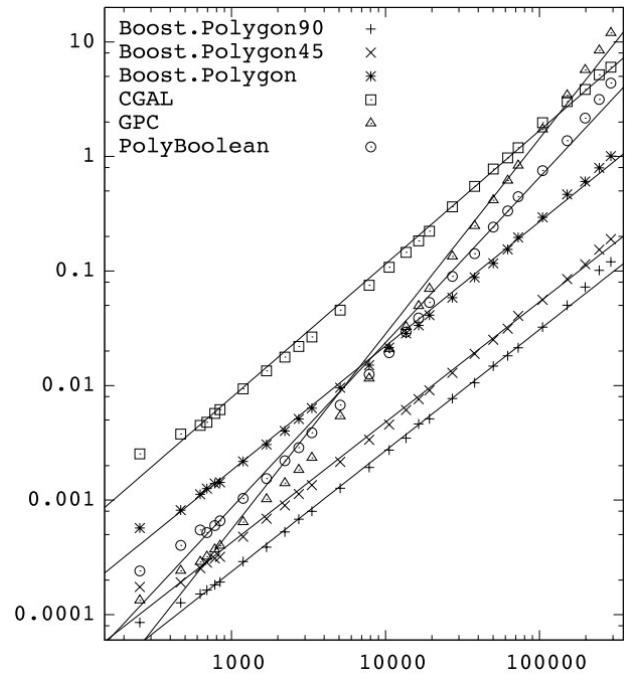


Fig. 5: Manhattan polygon clipping scalability comparison from Simonson [7]. The vertical axis represents the runtime in seconds using a logarithmic scale. The horizontal axis represents the number of input and output vertices combined using a logarithmic scale.

circumstances, we decided not to look into this library further.

Since Manhattan and 45-degree polygons are common in VLSI layout, Simonson repeated the experiment using Manhattan polygon data and compared the results. Figure 5 showed that their Manhattan polygon clipping implementation (labelled Boost.Polygon90) consistently performed 10 times faster than their general polygon clipping implementation. The Manhattan polygon clipper was consistently 2 times slower than their 45-degree polygon clipper (labelled Boost.Polygon45).

Another comprehensive comparison of eleven libraries was done by Rouge Modron [29] which clearly shows the advantages and drawbacks of these libraries.

VI. CONCLUSION

This paper’s goal is to provide a summary of the state-of-the-art in polygon clipping literature. This implies that the only information we could use was that which was available through studies, articles, and online research. In order to calculate and compare the runtimes, each algorithm should eventually be written in a low-level language (like C or a script language). The older algorithms may scale differently on newer technology because they were tested on hardware that was available at the time. This means that those algorithms do not account for recent hardware advancements in polygon clipping, such as enhanced multithreading, clusters, or even hardware for the algorithm’s particular calculations.

Algorithm	Clip Window	Subject Polygon	Holes	Degeneracies ^a	Loose Polygons	Complexity ^b	Reference
Sutherland-Hodgman	Irregular Convex ^c	Concave ^d	✗	✗	✗	*	[8]
Weiler-Atherton	Concave	Concave	✓	✓*	✓	***	[9]
Liang-Barsky	Rectangular	Concave*	✗*	✗*	✗*	*	[11]
Mailiot	Rectangular	Concave*	✗*	✗	✗*	**	[12]
Vatti	Complex ^e	Complex	✓	✓	✓	***	[13]
Slater-Barsky	Rectangular	Complex	✗*	✗*	✗*	*	[14]
Greiner-Hormann	Complex	Complex	✓	✗	✓	**	[15]
Zhang-Sabharwal	Rectangular	Concave*	✓*	✗*	✓*	*	[16]
Simonson	Complex*	Complex*	✓	✓	✓	**	[7]
Martinez-Rueda	Complex*	Complex	✓	✓	✓	**	[17]
Foster-Hormann	Complex	Complex	✓	✓	✓	**	[18]
Dimri	Rectangular	Concave	✗*	✓*	✓	*	[19]

^a Degeneracies refer to special cases or irregularities in polygons that require special handling, ^b Percieved complexity to implement

^c An irregular convex polygon is a polygon where each side is of unequal length, and all the interior angles are of unequal measure

^d Concave includes convex, ^e Complex includes concave and convex

TABLE III: Comparison of Polygon Clipping Algorithms. The entries marked with a * could not be validated and are educated guesses on the reference material gathered.

Library	Language(s)	Performance	Algorithm	Hole	Degeneracies	Loose Polygon	License	Reference
JTS	Java	Reliable	Vatti	✓	✓	✓	EPL-2.0, EDL-1.0	[21]
GEOS	C++	Reliable	Vatti	✓	✓	✓	LGPL	[22]
Shapely	Python	Reliable	Vatti	✓	✓	✓	BSD License	[23]
Boost.Geometry	C++	High performance	Weiler-Atherton	✓	✓	✓	BSL-1.0	[24]
Boost.Polygon	C++	High performance	Simonson	✓	✓	✓	BSL-1.0	[7]
CGAL	C++	High performance	Not specified	✓	✓	✓	LGPL	[27]
GPC	C	Accurate	Vatti	✓	✓	✓	MIT License	[25]
Clipper	C++, C#, Python, Delphi	High performance	Vatti	✓	✓	✓	BSL-1.0	[28]
Clipper 2	C++, C#, Delphi	High performance	Vatti	✓	✓	✓	BSL-1.0	[28]

TABLE IV: Comparison of Polygon Clipping Libraries. All these libraries can handle complex polygons as subjects or windows, except for CGAL, which can only handle simple polygons.

Furthermore, each algorithm may undoubtedly be improved further, making it challenging to obtain precise measurements. Future studies on specialised hardware accelerators for polygon clipping may also be conducted. It is required to utilise the same representation of a polygon and to provide a generic set of preconditions for the polygons to compare the various implementations of each algorithm. There is much more work to be done in the area of computer graphics. Each algorithm also has advantages and disadvantages. The Vatti algorithm is widely believed to be the most accessible, effective, and user-friendly algorithm. The enormous number of polygon clipping libraries that use the Vatti algorithm serve as confirmation of this. This is mainly a result of the Vatti algorithm's flexibility, feature set, and considerable research and improvements.

REFERENCES

- [1] W. M. Newman and R. F. Sproull, Eds., *Principles of Interactive Computer Graphics (2nd Ed.)*. USA: McGraw-Hill, Inc., 1979.
- [2] M. Cyrus and J. Beck, "Generalized two- and three-dimensional clipping," *Computers & Graphics*, vol. 3, no. 1, pp. 23–28, 1978. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0097849378900213>
- [3] Y.-D. Liang and B. A. Barsky, "A new concept and method for line clipping," *ACM Trans. Graph.*, vol. 3, no. 1, p. 1–22, jan 1984. [Online]. Available: <https://doi.org/10.1145/357332.357333>
- [4] C. Wylie, G. Romney, D. Evans, and A. Erdahl, "Half-tone perspective drawings by computer," in *Proceedings of the November 14-16, 1967, Fall Joint Computer Conference*, ser. AFIPS '67 (Fall). New York, NY, USA: Association for Computing Machinery, 1967, p. 49–58. [Online]. Available: <https://doi.org/10.1145/1465611.1465619>
- [5] J. Bouknight and K. Kelley, "An algorithm for producing half-tone computer graphics presentations with shadows and movable light sources," in *Proceedings of the May 5-7, 1970, Spring Joint Computer Conference*, ser. AFIPS '70 (Spring). New York, NY, USA: Association for Computing Machinery, 1970, p. 1–10. [Online]. Available: <https://doi.org/10.1145/1476936.1476938>
- [6] J. Warnock, "A hidden surface algorithm for computer generated half-tone pictures," *IEEE Transactions on Reliability - TR*, p. 35, 06 1969.
- [7] L. J. Simonson, "Industrial strength polygon clipping: A novel algorithm with applications in vlsi cad," *Computer-Aided Design*, vol. 42, no. 12, pp. 1189–1196, 2010. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0010448510001478>

- [8] I. E. Sutherland and G. W. Hodgman, "Reentrant polygon clipping," *Commun. ACM*, vol. 17, no. 1, p. 32–42, jan 1974. [Online]. Available: <https://doi.org/10.1145/360767.360802>
- [9] K. Weiler and P. Atherton, "Hidden surface removal using polygon area sorting," *SIGGRAPH Comput. Graph.*, vol. 11, no. 2, p. 214–222, jul 1977. [Online]. Available: <https://doi.org/10.1145/965141.563896>
- [10] A. Kilgour, "Unifying Vector and Polygon Algorithms for Scan Conversion and Clipping," in *EG 1987-Technical Papers*. Eurographics Association, 1987.
- [11] Y.-D. Liang and B. A. Barsky, "An analysis and algorithm for polygon clipping," *Commun. ACM*, vol. 26, no. 11, p. 868–877, nov 1983. [Online]. Available: <https://doi.org/10.1145/182.358439>
- [12] P.-G. Maillot, "A new, fast method for 2d polygon clipping: Analysis and software implementation," *ACM Trans. Graph.*, vol. 11, no. 3, p. 276–290, jul 1992. [Online]. Available: <https://doi.org/10.1145/130881.130894>
- [13] B. R. Vatti, "A generic solution to polygon clipping," *Commun. ACM*, vol. 35, no. 7, p. 56–63, jul 1992. [Online]. Available: <https://doi.org/10.1145/129902.129906>
- [14] M. Slater and B. A. Barsky, "2d line and polygon clipping based on space subdivision," *The Visual Computer*, vol. 10, no. 7, pp. 407–422, Aug 1994. [Online]. Available: <https://doi.org/10.1007/BF01900665>
- [15] G. Greiner and K. Hormann, "Efficient clipping of arbitrary polygons," *ACM Trans. Graph.*, vol. 17, no. 2, p. 71–83, apr 1998. [Online]. Available: <https://doi.org/10.1145/274363.274364>
- [16] M. Zhang and C. L. Sabharwal, "An efficient implementation of parametric line and polygon clipping algorithm," in *Proceedings of the 2002 ACM Symposium on Applied Computing*, ser. SAC '02. New York, NY, USA: Association for Computing Machinery, 2002, p. 796–800. [Online]. Available: <https://doi.org/10.1145/508791.508945>
- [17] F. Martínez, C. Ogayar, J. R. Jiménez, and A. J. Rueda, "A simple algorithm for boolean operations on polygons," *Advances in Engineering Software*, vol. 64, pp. 11–19, 2013. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0965997813000379>
- [18] E. L. Foster, K. Hormann, and R. T. Popa, "Clipping simple polygons with degenerate intersections," *Computers & Graphics: X*, vol. 2, p. 100007, 2019. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S259014861930007X>
- [19] S. C. Dimri, U. K. Tiwari, and M. Ram, "An efficient algorithm to clip a 2d-polygon against a rectangular clip window," *Applied Mathematics-A Journal of Chinese Universities*, vol. 37, no. 1, pp. 147–158, Mar 2022. [Online]. Available: <https://doi.org/10.1007/s11766-022-4556-0>
- [20] L. Vogt, Y. Zimmermann, and J. Schilp, "Computing gripping points in 2d parallel surfaces via polygon clipping," in *Annals of Scientific Society for Assembly, Handling and Industrial Robotics 2021*, T. Schüppstuhl, K. Tracht, and A. Raatz, Eds. Cham: Springer International Publishing, 2022, pp. 101–112.
- [21] JTS contributors, "Jts frequently asked questions," September 2020. [Online]. Available: <https://locationtech.github.io/jts/jts-faq.html#E1> (Accessed 01.06.2023).
- [22] GEOS contributors, *GEOS coordinate transformation software library*, Open Source Geospatial Foundation, 2021. [Online]. Available: <https://libgeos.org/>
- [23] S. Gillies, C. van der Wel, J. Van den Bossche, M. W. Taves, J. Arnott, B. C. Ward *et al.*, "Shapely," Jan. 2023. [Online]. Available: <https://doi.org/10.5281/zenodo.7583915>
- [24] B. Gehrels, "Barend's blog (boost.geometry, programming, music, languages): Intersections (1)," December 2010. [Online]. Available: <https://barendgehrels.blogspot.com/2010/12/intersections-1.html> (Accessed 01.06.2023).
- [25] A. Murta, "GPC: General Polygon Clipper library," Astrophysics Source Code Library, record ascl:1512.006, p. ascl:1512.006, Dec. 2015, provided by the SAO/NASA Astrophysics Data System. [Online]. Available: <https://ui.adsabs.harvard.edu/abs/2015ascl.soft12006M>
- [26] A. Murta, "A general polygon clipping library," 2011. [Online]. Available: <https://web.archive.org/web/20110920184218/http://www.cs.man.ac.uk/~toby/alan/software/gpc.html> (Accessed 01.06.2023).
- [27] The CGAL Project, *CGAL User and Reference Manual*, 5.5.2 ed. CGAL Editorial Board, 2023. [Online]. Available: <https://doc.cgal.org/5.5.2/Manual/packages.html>
- [28] A. Johnson, "Clipper2 - polygon clipping and offsetting," May 2023. [Online]. Available: http://www.angusj.com/clipper2/Docs/_Body.htm (Accessed 01.06.2023).
- [29] R. Modron, "The crag of rogue modron: Polygon clipping: a wrapper, a benchmark," March 2013. [Online]. Available: <https://rogue-modron.blogspot.com/2011/04/polygon-clipping-wrapper-benchmark.html> (Accessed 01.06.2023).